

Express.JS CheatSheet





TABLE OF CONTENTS

1. Introduction to Express.js

- What is Express.js?
- History and Features
- Why Use Express?
- Express vs Node.js HTTP Module

2. Environment Setup

- Installing Node.js and npm
- Creating a New Express App
- Understanding Project Structure
- Using nodemon for Auto-Restart
- Installing Postman or Thunder Client for Testing

3. Basic Express Server

- Creating a Basic Server
- Using app.listen()
- Hello World Example
- Port Configuration with .env

4. Routing in Express

- Basic Routes (app.get(), app.post(), etc.)
- Route Parameters (/user/:id)
- Query Parameters (?name=value)
- Chaining Route Handlers
- app.route() for Cleaner Code

5. Middleware in Express

- What is Middleware?
- Types of Middleware
- Built-in Middleware (express.json(), express.urlencoded())
- Third-party Middleware (morgan, cors, helmet)
- Creating Custom Middleware
- Middleware Flow with next()

6. Request and Response Objects

- req.params, req.query, req.body, req.headers
- res.send(), res.json(), res.status()
- Chaining Response Methods
- Sending Files or HTML



TABLE OF CONTENTS

7. Working with Forms and JSON

- Handling Form Data
- Parsing URL-encoded Data
- Handling JSON Requests
- Form Submission Example

8. Serving Static Files

- Using `express.static()`
- Serving Images, CSS, JS
- Custom Static Folder Configuration

9. Express Router

- What is a Router?
- Creating Route Modules
- Using `express.Router()`
- Grouping and Mounting Routes

10. Error Handling

- Default Error Handling
- 404 Not Found Route
- Custom Error Middleware
- Using `next(err)` for Custom Errors
- Centralized Error Handling

11. Templating Engines

- What is a Templating Engine?
- Installing and Using EJS
- Passing Data to Views
- Pug and Handlebars (Alternatives)

12. Connecting to MongoDB

- Installing and Configuring Mongoose
- Creating Models and Schemas
- Connecting to MongoDB Atlas
- Basic CRUD Operations

13. Environment Variables

- Why Use `.env` File?
- Using `dotenv` Package
- Storing Secrets (`DB_URL`, `JWT_SECRET`)
- Accessing Env Vars in Code



TABLE OF CONTENTS

14. RESTful API Design

- What is REST API?
- Creating Routes for CRUD
- Following REST Conventions
- Versioning APIs

15. Authentication with JWT

- What is JWT?
- Generating and Verifying Tokens
- Login and Register Routes
- Protected Routes Middleware
- Token Expiration and Refresh Tokens

16. File Uploads

- Installing and Using multer
- Single File Upload
- Multiple File Upload
- Serving Uploaded Files

17. Input Validation and Sanitization

- Using express-validator
- Creating Validation Middleware
- Custom Error Messages
- Avoiding Injection Attacks

18. Security in Express Apps

- Using helmet for Security Headers
- CORS Setup with cors
- Rate Limiting with express-rate-limit
- XSS and CSRF Protection

19. Logging and Debugging

- Using morgan for Request Logging
- Debugging with Console Logs
- Debugging with VS Code
- Postman or Thunder Client for Testing APIs

20. Express App Deployment

- Production Environment Setup
- Deploying to Vercel, Render, Railway
- Using PM2 for Background Services
- Reverse Proxy with NGINX (Optional)



TABLE OF CONTENTS

21. Performance Optimization

- Enabling GZIP Compression
- Caching Responses
- Async/Await for Non-blocking Code
- Optimizing Middleware Chain

22. Using Express with SQL

- Introduction to Sequelize ORM
- Connecting to MySQL or PostgreSQL
- Defining Models
- Performing SQL CRUD Operations

23. Using Express with TypeScript

- Setting Up TypeScript in Express
- Typing req, res, and next
- TSConfig Best Practices
- Folder Structure in TS Projects

24. Advanced API Features

- Pagination
- Filtering and Searching
- Sorting with Query Parameters
- Handling Large Data Sets

25. Role-Based Access Control (RBAC)

- Admin vs User Role Checks
- Middleware for Role Verification
- Protecting Routes Based on Roles

26. Testing Express Applications

- Unit Testing with Jest
- Integration Testing with Supertest
- Testing Routes and Controllers
- Mocking Database and JWT

27. WebSockets with Express

- What are WebSockets?
- Using socket.io with Express
- Real-time Messaging Example
- Socket Events and Rooms



TABLE OF CONTENTS

28. Code Structure and Best Practices

- Folder Structure for Large Projects
- Service vs Controller Layer
- Using Async/Await Everywhere
- Avoiding Callback Hell

29. Popular Express Packages

- cors, helmet, morgan, dotenv
- express-validator, multer, jsonwebtoken
- compression, cookie-parser, express-rate-limit

30. Common Mistakes Beginners Make in Express.js

- Forgetting to Start the Server
- Not Using Middleware Correctly
- Not Handling Errors Properly
- Mixing Logic Inside Routes
- Hardcoding Secrets (Like Passwords or Keys)
- Not Using .env Files
- Forgetting to Parse Request Body
- Using Wrong HTTP Methods
- Ignoring Asynchronous Code Issues
- Not Using Route Files and Routers
- Missing Return Statements in Middleware
- Not Validating User Input
- Not Handling 404 Errors
- Not Securing API with Helmet or CORS
- Not Using Status Codes Properly
- Not Closing Database Connections
- Forgetting to Use next() in Middleware
- Overusing Global Middleware
- Not Handling Large Files Carefully
- Not Using try/catch in Async Functions



1. INTRODUCTION TO EXPRESS.JS

1.1 What is Express.js?

- Express.js is a tool that helps us build websites and web apps easily using JavaScript and Node.js.
- It makes it simple to create web pages, send data, and handle user requests.
- Think of Express as a smart assistant that helps your website work better and faster.

1.2 History and Features

- Express.js was created by a developer named TJ Holowaychuk in 2010.
- It is built on top of Node.js and helps developers write less code to do more work.

Main Features:

- Easy to use for building websites and APIs
- Handles different types of requests like GET, POST, PUT, DELETE
- Works with any front-end (like HTML, CSS, React, etc.)
- Can connect with databases like MongoDB and MySQL

1.3 Why Use Express?

- Express saves time. Without Express, we would need to write a lot of code to do simple things like showing a page or sending data.

With Express:

- The code is shorter and cleaner
- We can manage routes (paths like /home or /about) easily
- It works well with many other tools

1.4 Express vs Node.js HTTP Module

- Node.js has a built-in module called http to create servers. But it is very basic and takes more time to write.

Node.js http module:

- You must write everything manually.
- Not beginner-friendly.

Express.js:

- Easier to use.
- Helps you build faster with less code.

Example:

- To create a simple server with Node.js http takes 10–15 lines.
- With Express, it only takes 2–3 lines.



2. ENVIRONMENT SETUP

2.1 Installing Node.js and npm

- To use Express, we first need Node.js.
- Node.js is like the engine that runs your JavaScript on the computer (not just in the browser).

Steps:

- Go to <https://nodejs.org>
- Download and install the LTS version
- After install, open your terminal or command prompt
- Type: `node -v` and `npm -v` to check if it's installed

2.2 Creating a New Express App

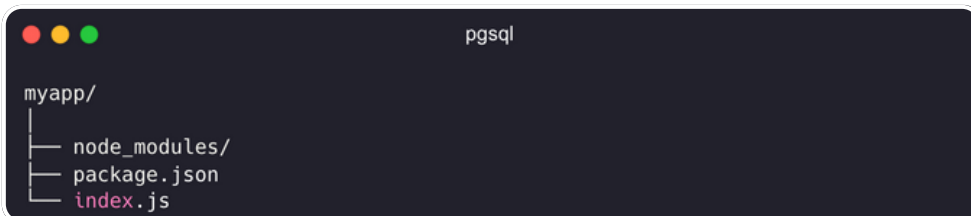
- Once Node.js is installed, we can create a new Express app.

Steps:

- Open terminal
- Make a new folder: `mkdir myapp`
- Go inside the folder: `cd myapp`
- Run: `npm init -y` (this creates a `package.json` file)
- Install Express: `npm install express`

2.3 Understanding Project Structure

- After creating the app, your folder might look like this:



- `node_modules/`: All the tools we install
- `package.json`: Keeps track of installed tools
- `index.js`: Our main app file

2.4 Using nodemon for Auto-Restart

- Every time we change code, we don't want to restart the server manually.
- So we use nodemon to auto-restart.

Install it:

- `npm install -g nodemon`

Run your app:

- Instead of `node index.js`, use:

`nodemon index.js`



2. ENVIRONMENT SETUP

2.5 Installing Postman or Thunder Client for Testing

- When building APIs, we need to test them.

Instead of using a browser, we can use tools like:

- Postman: A free app for testing APIs
- Thunder Client: A VS Code extension for testing APIs

Use them to:

- Send requests (GET, POST, etc.)
- See responses
- Test your routes easily



3. BASIC EXPRESS SERVER

3.1 Creating a Basic Server

- Create a file called index.js and write this code:

```
js
const express = require('express');
const app = express();

app.get('/', (req, res) => {
  res.send('Hello World');
});

app.listen(3000, () => {
  console.log('Server is running on port 3000');
});
```

- This code creates a basic web server.

3.2 Using app.listen()

- app.listen() tells your app to start running on a port.

```
js
app.listen(3000, () => {
  console.log('Server started');
});
```

- This means: "Start the server and listen on port 3000."

3.3 Hello World Example

- The line:

```
js
app.get('/', (req, res) => {
  res.send('Hello World');
});
```

means:

- If someone visits the homepage ("/"), send them the message "Hello World".

3.4 Port Configuration with .env

- Instead of writing the port directly, we can store it in a .env file.

Steps:

1. Install dotenv: npm install dotenv
2. Create .env file:

```
ini
PORT=3000
```



3. BASIC EXPRESS SERVER

3. Change your code:

```
js

require('dotenv').config();
const express = require('express');
const app = express();

const PORT = process.env.PORT;

app.get('/', (req, res) => {
  res.send('Hello World');
});

app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```



4. ROUTING IN EXPRESS

4.1 Basic Routes (app.get(), app.post(), etc.)

Routes tell your app what to do when someone visits a certain address.

- app.get() handles when someone asks for data (like opening a webpage).
- app.post() handles when someone sends data (like filling a form).

Example:

```
js
app.get('/', (req, res) => {
  res.send('Welcome to Home Page');
});

app.post('/submit', (req, res) => {
  res.send('Form submitted');
});
```

4.2 Route Parameters (/user/:id)

- Sometimes you want to get information from the URL.

Example: /user/5

- Here, 5 is the ID of the user.

You write this in Express like:

```
js
app.get('/user/:id', (req, res) => {
  const userId = req.params.id;
  res.send(`User ID is ${userId}`);
});
```

4.3 Query Parameters (?name=value)

- Query parameters come after a question mark ? in the URL.

Example: /search?name=Rudra

You get this in Express as:

```
js
app.get('/search', (req, res) => {
  const name = req.query.name;
  res.send(`Searching for ${name}`);
});
```

4.4 Chaining Route Handlers

- You can add multiple functions to handle a route step-by-step.

Example:

```
js
app.get('/example', (req, res, next) => {
  console.log('First handler');
  next(); // move to next handler
}, (req, res) => {
  res.send('Second handler response');
});
```




4. ROUTING IN EXPRESS

4.5 app.route() for Cleaner Code

- Instead of writing `app.get()`, `app.post()` separately for the same path, you can chain them.

Example:

```
js
app.route('/book')
  .get((req, res) => {
    res.send('Get a book');
  })
  .post((req, res) => {
    res.send('Add a book');
  });
```



5. MIDDLEWARE IN EXPRESS

5.1 What is Middleware?

- Middleware is like a helper that runs between the request and response.
- It can do things like check data, log info, or change requests.

5.2 Types of Middleware

- Built-in: Comes with Express
- Third-party: Made by others and you install them
- Custom: Made by you for your app's needs

5.3 Built-in Middleware (`express.json()`, `express.urlencoded()`)

- These help your app read data sent by the user.

```
js
app.use(express.json()); // to read JSON data
app.use(express.urlencoded({ extended: true })); // to read form data
```

5.4 Third-party Middleware (`morgan`, `cors`, `helmet`)

- `morgan`: Logs info about requests
- `cors`: Lets your app talk to other websites
- `helmet`: Helps keep your app safe

You install with npm, like:

- `npm install morgan cors helmet`

Then use in your app:

```
js
const morgan = require('morgan');
const cors = require('cors');
const helmet = require('helmet');

app.use(morgan('dev'));
app.use(cors());
app.use(helmet());
```

5.5 Creating Custom Middleware

- You can make your own middleware function:

```
js
const myMiddleware = (req, res, next) => {
  console.log('Custom middleware running');
  next(); // continue to next step
};

app.use(myMiddleware);
```



5. MIDDLEWARE IN EXPRESS

5.6 Middleware Flow with next()

- `next()` tells Express to move to the next middleware or route handler.
- If you forget `next()`, the request will stop there.



6. REQUEST AND RESPONSE OBJECTS

6.1 req.params, req.query, req.body, req.headers

- req.params: Data in URL (like /user/:id)
- req.query: Data after ? in URL (like ?name=Rudra)
- req.body: Data sent in form or JSON (needs middleware)
- req.headers: Extra info sent by browser or client

6.2 res.send(), res.json(), res.status()

- res.send() sends text or HTML to the user
- res.json() sends data as JSON (useful for APIs)
- res.status() sets the HTTP status code (like 200 for success or 404 for not found)

Example:

```
js
res.status(200).send('Success');
res.status(404).send('Page not found');
res.json({ message: 'Hello JSON' });
```

6.3 Chaining Response Methods

- You can chain methods for clean code:

```
js
res.status(200).json({ message: 'OK' });
```

6.4 Sending Files or HTML

- To send a file or HTML page:

```
js
app.get('/home', (req, res) => {
  res.sendFile(__dirname + '/index.html');
});
```

“__dirname” means current folder.



7. WORKING WITH FORMS AND JSON

7.1 Handling Form Data

- When users fill out a form on a website and click "submit", the browser sends that data to the server.
- To collect this data in Express, we need to set up a middleware to read it properly.

7.2 Parsing URL-encoded Data

- HTML forms usually send data in a format called URL-encoded.

To read this in Express:

```
js
app.use(express.urlencoded({ extended: true }));
```

- Now, when someone submits a form like:

```
html
<form action="/submit" method="POST">
  <input name="name" />
  <button>Submit</button>
</form>
```

- You can access the name field like this:

```
js
app.post('/submit', (req, res) => {
  const name = req.body.name;
  res.send(`You submitted: ${name}`);
});
```

7.3 Handling JSON Requests

- When sending data using JavaScript (like from React or AJAX), it's often sent as JSON.
- To read JSON data in Express:

```
js
app.use(express.json());
```

Example:

```
js
app.post('/json', (req, res) => {
  const data = req.body;
  res.json({ received: data });
});
```



7. WORKING WITH FORMS AND JSON

7.4 Form Submission Example

HTML Form:

```
html

<form action="/contact" method="POST">
  <input name="email" />
  <button>Send</button>
</form>
```

Express Code:

```
js

app.use(express.urlencoded({ extended: true }));

app.post('/contact', (req, res) => {
  const email = req.body.email;
  res.send(`Thanks! We got your email: ${email}`);
});
```



8. SERVING STATIC FILES

8.1 Using `express.static()`

Static files are things like:

- HTML files
- CSS stylesheets
- JavaScript scripts
- Images

To serve these, use:

```
js
app.use(express.static('public'));
```

Now if you have:

```
arduino
public/
├── index.html
├── style.css
└── image.png
```

You can open them like:

- `/index.html`
- `/style.css`
- `/image.png`

8.2 Serving Images, CSS, JS

- Put all your static files in a folder (like `public`) and Express will serve them automatically.

Example:

```
js
app.use(express.static('public'));
```

Your HTML can now use:

```
html
<link rel="stylesheet" href="/style.css" />

```

8.3 Custom Static Folder Configuration

- If you want to change the folder name or use a different path:

```
js
app.use('/static', express.static('myfolder'));
```

Now you access files like `/static/file.png` instead of `/file.png`.



9. EXPRESS ROUTER

9.1 What is a Router?

- A router is a mini version of your Express app.
- It helps you organize your routes better, especially when your app grows big.

9.2 Creating Route Modules

- You can create a new file for your routes.

Example: users.js

```
js

const express = require('express');
const router = express.Router();

router.get('/', (req, res) => {
  res.send('All Users');
});

router.get('/:id', (req, res) => {
  res.send(`User ID: ${req.params.id}`);
});

module.exports = router;
```

9.3 Using express.Router()

- In your main file (like index.js or app.js):

```
js

const express = require('express');
const app = express();

const usersRoute = require('./users');

app.use('/users', usersRoute);
```

Now:

- /users → shows "All Users"
- /users/5 → shows "User ID: 5"

9.4 Grouping and Mounting Routes

You can group all your routes by topic:

- /users → for users
- /products → for products
- /admin → for admin pages

This makes your app cleaner and easier to manage.



10. ERROR HANDLING

10.1 Default Error Handling

- When something goes wrong in your Express app (like a mistake in code), Express will show a basic error message in the browser.
- For example:** If someone visits a page that doesn't exist like `/wrongpage`, Express shows:

```
bash  
Cannot GET /wrongpage
```

- This is the default error handling by Express.

10.2 404 Not Found Route

- A 404 error means the page the user is looking for is not found.
- To show a custom message like "Oops! Page Not Found", we can add this at the bottom of our file:

```
js  
  
app.use((req, res) => {  
  res.status(404).send("Oops! Page Not Found.");  
});
```

This tells Express:

- "If no other route matched, just show this message."

10.3 Custom Error Middleware

- Sometimes, we want to handle errors in our own way — like showing a friendly message or logging it.
- Express lets us do this with special error-handling middleware. It has 4 parameters: (err, req, res, next)

Example:

```
js  
  
app.use((err, req, res, next) => {  
  console.error(err.stack);  
  res.status(500).send("Something went wrong!");  
});
```

- If there's a bug or mistake, this will catch it and show a custom message.

10.4 Using next(err) for Custom Errors

- We can create our own errors and send them using `next()`.

```
js  
  
app.get('/error', (req, res, next) => {  
  const myError = new Error("This is a custom error");  
  next(myError); // send to error handler  
});
```

Then Express will send this to the custom error middleware we made.



10. ERROR HANDLING

10.5 Centralized Error Handling

- Instead of writing error handling in every route, we put one big error handler at the bottom of our app.

```
js
app.use((err, req, res, next) => {
  res.status(500).json({ message: err.message });
});
```

This way, all errors go through this one place. It's neat and clean!



11. TEMPLATING ENGINES

11.1 What is a Templating Engine?

- A templating engine helps us mix HTML with JavaScript data.

Example: Instead of writing just this:

```
html
<h1>Hello</h1>
```

We can write:

```
html
<h1>Hello <%= name %></h1>
```

And if name = "Rudra", the browser will show:

```
nginx
Hello Rudra
```

11.2 Installing and Using EJS (a template engine)

To use EJS:

1. Install it:

```
bash
npm install ejs
```

2. Tell Express to use EJS:

```
js
app.set('view engine', 'ejs');
```

3. Create a folder called views and make a file home.ejs:

```
html
<h1>Welcome <%= username %></h1>
```

4. Render it from Express:

```
js
app.get('/', (req, res) => {
  res.render('home', { username: 'Rudra' });
});
```

11.3 Passing Data to Views

- You can send data from your route to the EJS file like this:

```
js
res.render('profile', { name: 'Alice', age: 10 });
```



11. TEMPLATING ENGINES

11.3 Passing Data to Views

- Then inside profile.ejs, you can use:

```
html
<p>Name: <%= name %></p>
<p>Age: <%= age %></p>
```

11.4 Pug and Handlebars (Other Options)

Besides EJS, there are other templating engines too:

- Pug – uses indentation instead of HTML tags.
- Handlebars (hbs) – uses {{name}} instead of <%= name %>

All do the same job: mix HTML with your data.



12. CONNECTING TO MONGODB

12.1 Installing and Configuring Mongoose

- Mongoose is a tool that helps us talk to MongoDB easily.

Install it:

```
bash
npm install mongoose
```

Connect to MongoDB:

```
js
const mongoose = require('mongoose');

mongoose.connect('mongodb://localhost:27017/myapp')
  .then(() => console.log("MongoDB Connected"))
  .catch(err => console.log(err));
```

12.2 Creating Models and Schemas

- A schema tells how our data should look.

```
js
const userSchema = new mongoose.Schema({
  name: String,
  age: Number
});
```

- A model lets us use this schema to make, find, and delete users.

```
js
const User = mongoose.model('User', userSchema);
```

12.3 Connecting to MongoDB Atlas (Cloud Database)

- Go to: <https://cloud.mongodb.com>
- Make a free account
- Create a cluster
- Copy the connection string like:

```
bash
mongodb+srv://username:password@cluster.mongodb.net/myapp
```

- Use it in your mongoose.connect() like this:

```
js
mongoose.connect(process.env.DB_URL);
```



12. CONNECTING TO MONGODB

12.4 Basic CRUD with Mongoose

Create:

```
js  
  
const newUser = new User({ name: 'Rudra', age: 19 });  
newUser.save();
```

Read:

```
js  
  
User.find().then(data => console.log(data));
```

Update:

```
js  
  
User.updateOne({ name: 'Rudra' }, { age: 20 });
```

Delete:

```
js  
  
User.deleteOne({ name: 'Rudra' });
```



13. ENVIRONMENT VARIABLES

13.1 Why Use .env File?

- We don't want to write passwords or secret keys in our code.
- We put them in a hidden file called .env.

Example:

```
DB_URL=mongodb+srv://user:pass@cluster.mongodb.net/mydb
PORT=3000
JWT_SECRET=superSecret123
```

13.2 Using dotenv Package

1. Install:

```
npm install dotenv
```

2. Use at the top of your file:

```
const db = process.env.DB_URL;
const port = process.env.PORT;

mongoose.connect(db);
app.listen(port, () => console.log(`Server running on ${port}`));
```

- This way, your secrets stay safe and your code is clean.



14. RESTFUL API DESIGN

14.1 What is a REST API?

- REST stands for Representational State Transfer — a fancy way of saying:
- “We use simple rules to send and get data from a server using HTTP.”

In a REST API:

- We use URLs like /users, /products
- And methods like:
 - GET (to get data)
 - POST (to add data)
 - PUT or PATCH (to update data)
 - DELETE (to delete data)

Imagine you have a toy box (database), and you want to:

- See toys → GET /toys
- Add a toy → POST /toys
- Update a toy → PUT /toys/:id
- Delete a toy → DELETE /toys/:id

14.2 Creating Routes for CRUD

- **CRUD** = Create, Read, Update, Delete

Here's a simple example using Express:

```
js

const express = require('express');
const app = express();
app.use(express.json());

const users = [];

app.get('/users', (req, res) => {
  res.json(users); // Read
});

app.post('/users', (req, res) => {
  users.push(req.body); // Create
  res.status(201).send("User added");
});

app.put('/users/:id', (req, res) => {
  users[req.params.id] = req.body; // Update
  res.send("User updated");
});

app.delete('/users/:id', (req, res) => {
  users.splice(req.params.id, 1); // Delete
  res.send("User deleted");
});
```

14.3 Following REST Conventions

Some basic REST rules:

- Use nouns in URLs, not verbs
- ✓ /users (not /getUsers)
- Use correct HTTP methods (GET, POST, PUT, DELETE)
- Don't use too many query strings
- ✓ /products/123 instead of /getProduct?id=123



14. RESTFUL API DESIGN

14.4 Versioning APIs

- When your API changes over time, you don't want to break older apps.
- So use versions like:

```
/api/v1/users  
/api/v2/users
```

- That way, version 1 and 2 can both work independently.



15. AUTHENTICATION WITH JWT

15.1 What is JWT?

- JWT stands for JSON Web Token.
- It's like a digital ID card.
- When a user logs in, we give them a token.
- Then they send this token every time they want to access protected data.

15.2 Generating and Verifying Tokens

Install JWT:

```
bash

npm install jsonwebtoken
```

Create a token:

```
js

const jwt = require('jsonwebtoken');
const token = jwt.sign({ userId: 123 }, 'secretKey');
```

Verify the token:

```
js

jwt.verify(token, 'secretKey');
```

15.3 Login and Register Routes

Example:

```
js

const users = [];

app.post('/register', (req, res) => {
  const { username, password } = req.body;
  users.push({ username, password });
  res.send("User registered");
});

app.post('/login', (req, res) => {
  const { username, password } = req.body;
  const user = users.find(u => u.username === username && u.password === password);
  if (!user) return res.status(401).send("Invalid login");
  const token = jwt.sign({ username }, 'secretKey');
  res.json({ token });
});
```

15.4 Protected Routes Middleware

- Only logged-in users (with a token) can access these routes.



15. AUTHENTICATION WITH JWT

15.4 Protected Routes Middleware

```
js

function authMiddleware(req, res, next) {
  const token = req.headers.authorization?.split(" ")[1];
  if (!token) return res.status(401).send("Token missing");

  try {
    const decoded = jwt.verify(token, 'secretKey');
    req.user = decoded;
    next();
  } catch (err) {
    res.status(403).send("Invalid token");
  }
}

app.get('/secret', authMiddleware, (req, res) => {
  res.send("This is a secret page!");
});
```

15.5 Token Expiration and Refresh Tokens

- You can make tokens expire:

```
js

const token = jwt.sign({ username }, 'secretKey', { expiresIn: '1h' });
```

- After 1 hour, it won't work. You can use a refresh token (a second longer-living token) to get a new one.



16. FILE UPLOADS

16.1 Installing and Using multer

- Multer helps Express handle file uploads.

Install it:

```
bash
npm install multer
```

Basic setup:

```
js
const multer = require('multer');
const upload = multer({ dest: 'uploads/' }); // files saved in uploads/
folder
```

16.2 Single File Upload

```
js
app.post('/upload', upload.single('file'), (req, res) => {
  res.send("File uploaded!");
});
```

- Use a tool like Postman to test by sending a file in form-data with key name file.

16.3 Multiple File Upload

```
js
app.post('/uploads', upload.array('files', 5), (req, res) => {
  res.send("Multiple files uploaded!");
});
```

- This allows up to 5 files at once.

16.4 Serving Uploaded Files

- You can make uploaded files publicly available:

```
js
app.use('/uploads', express.static('uploads'));
```

- Now, if someone uploaded photo.jpg, they can see it at:

```
bash
http://localhost:3000/uploads/photo.jpg
```



17. INPUT VALIDATION AND SANITIZATION

17.1 Using express-validator

- Sometimes users give bad or dangerous input (like wrong email or scripts).
- We can check and clean their data using a package called:

```
bash
npm install express-validator
```

- Then in your code:

```
js
const { check, validationResult } = require('express-validator');
```

17.2 Creating Validation Middleware

- This is how we add simple rules to check user input:

```
js
app.post('/signup', [
  check('email').isEmail(),
  check('password').isLength({ min: 5 })
], (req, res) => {
  const errors = validationResult(req);
  if (!errors.isEmpty()) return res.status(400).json({ errors: errors.array() });

  res.send("Signup successful!");
});
```

Here we are saying:

- Email should be valid
- Password should be at least 5 letters

17.3 Custom Error Messages

- You can show messages that help users understand the problem:

```
js
check('email')
  .isEmail()
  .withMessage('Please enter a valid email');
```

Now if someone enters abc as email, they will see:

- "Please enter a valid email"

17.4 Avoiding Injection Attacks

Some people try to hack by writing things like:

```
js
email = "admin@example.com' OR 1=1"
```



17. INPUT VALIDATION AND SANITIZATION

17.4 Avoiding Injection Attacks

- That's dangerous.
- To protect your app:
- Always sanitize inputs using:

```
check('username').trim().escape();
```

- This removes spaces and special characters.

Also:

- Never use raw user input in your database without checking
- Use tools like Mongoose that escape input safely



18. SECURITY IN EXPRESS APPS

18.1 Using helmet for Security Headers

- Helmet adds small protections to your server to block bad behavior.

Install:

```
bash
npm install helmet
```

Use it:

```
js
const helmet = require('helmet');
app.use(helmet());
```

It sets HTTP headers that protect against:

- Clickjacking
- Script attacks
- MIME sniffing, etc.

18.2 CORS Setup with cors

- CORS = Cross-Origin Resource Sharing
- Sometimes you want your server to allow requests only from your app, not random websites.

Install:

```
bash
npm install cors
```

Use it:

```
js
const cors = require('cors');
app.use(cors()); // Allow all origins

// Or limit it:
app.use(cors({
  origin: 'https://myfrontend.com'
}));
```

18.3 Rate Limiting with express-rate-limit

- To stop bots or hackers from sending too many requests.

Install:

```
bash
npm install express-rate-limit
```



18. SECURITY IN EXPRESS APPS

18.3 Rate Limiting with express-rate-limit

Use it:

```
js

const rateLimit = require('express-rate-limit');

const limiter = rateLimit({
  windowMs: 1 * 60 * 1000, // 1 minute
  max: 5 // only 5 requests allowed per minute
});

app.use(limiter);
```

18.4 XSS and CSRF Protection

- **XSS (Cross-site scripting)** = Bad people try to send scripts like `<script>stealData()</script>`.
 - Avoid this with helmet, express-validator, and escaping input.
- **CSRF (Cross-site request forgery)** = Someone tricks you into doing an action without permission.
 - Use the csrf package for CSRF protection if you're using sessions or forms:

```
bash

npm install csrf
```

Use:

```
js

const csrf = require('csrf');
app.use(csrf());
```




19. LOGGING AND DEBUGGING

19.1 Using morgan for Request Logging

- Morgan is a tool that shows who visited your app, what route they used, and their IP.

Install:

```
bash
npm install morgan
```

Use:

```
js
const morgan = require('morgan');
app.use(morgan('dev'));
```

This will log things like:

```
bash
GET /users 200 12ms
POST /login 401 8ms
```

19.2 Debugging with Console Logs

- Sometimes things break. You can print stuff using:

```
js
console.log("User ID:", req.params.id);
console.error("Something went wrong");
```

You can check what's going wrong by printing it in the terminal.

19.3 Debugging with VS Code

If you're using Visual Studio Code, you can:

- Set breakpoints by clicking beside line numbers.
- Use the Debugger panel to run the code step-by-step.
- Hover over variables to see their value.

This helps find bugs easily!

19.4 Postman or Thunder Client for Testing APIs

- Postman and Thunder Client (VS Code extension) let you test your API easily.

You can:

- Send POST, GET, DELETE requests
- Add headers (like tokens)
- Upload files
- See full response (status, body, headers)

It's like using a remote control to check your API.



20. EXPRESS APP DEPLOYMENT

20.1 Production Environment Setup

- Before sharing your app with the world, you need to prepare it for production.

Basic things to do:

- Keep secrets in .env (like DB_URL)
- Don't show error messages to users
- Use tools like helmet, cors, and express-rate-limit for safety

Also, make sure you have:

```
js
app.listen(process.env.PORT || 3000, () => {
  console.log("Server started");
});
```

20.2 Deploying to Vercel, Render, Railway

- You can host your app online without buying a server.
- [Vercel](#) (Best for Frontend like React, not ideal for Express backend)
- Use it mainly for frontend apps
- Not best for Node.js server (they sleep or timeout)
- [Render](#)
- Great for Express apps
- [Steps:](#)
 - a. Push code to GitHub
 - b. Create new Web Service in Render
 - c. Connect your GitHub repo
 - d. Set start script in package.json and PORT
- [Railway](#)
- Super easy for Express + MongoDB/MySQL
- [Steps:](#)
 1. Login at : <https://railway.app>
 2. Click "New Project" → "Deploy from GitHub"
 3. Set up environment variables
 4. Done! App goes live

20.3 Using PM2 for Background Services

If you're using a VPS or real server like AWS or DigitalOcean:

- [Install PM2:](#)

```
bash
npm install pm2 -g
```



20. EXPRESS APP DEPLOYMENT

20.3 Using PM2 for Background Services

- Run your app:

```
bash
pm2 start index.js
```

- This keeps your app running forever, even if it crashes.
- You can even use:

```
bash
pm2 startup
pm2 save
```

- To auto-start on system reboot.

20.4 Reverse Proxy with NGINX (Optional)

If you're using Linux server, you can use NGINX to:

- Forward traffic from <https://yourdomain.com> to your app running on port 3000
- Handle HTTPS with SSL

Basic NGINX config:

```
nginx
server {
    listen 80;
    server_name yourdomain.com;

    location / {
        proxy_pass http://localhost:3000;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```



21. PERFORMANCE OPTIMIZATION

21.1 Enabling GZIP Compression

- Use compression middleware to make responses smaller.

Install:

```
bash
npm install compression
```

Use:

```
js
const compression = require('compression');
app.use(compression());
```

- Now, your data loads faster!

21.2 Caching Responses

- For pages that don't change often, use cache:

```
js
app.get('/static-data', (req, res) => {
  res.set('Cache-Control', 'public, max-age=600'); // 10 mins
  res.json({ message: "This is cached data" });
});
```

- This means the browser will reuse the same response for 10 minutes.

21.3 Async/Await for Non-blocking Code

- Always use async functions when dealing with things like database or API calls:

```
js
app.get('/users', async (req, res) => {
  try {
    const users = await getUsersFromDB();
    res.json(users);
  } catch (err) {
    res.status(500).send("Something broke");
  }
});
```

- It helps your server keep running smoothly even when waiting for something.

21.4 Optimizing Middleware Chain

- Don't add middleware you don't need.

Example:



21. PERFORMANCE OPTIMIZATION

21.4 Optimizing Middleware Chain

- Don't add middleware you don't need.

Example:

```
js

// Bad: Adds to every request, even GET
app.use(express.json());

// Better: Only adds when needed
app.post('/form', express.json(), (req, res) => {
  res.send("Form received");
});
```

- Also: use `app.use()` in correct order (like `helmet`, `cors` first, routes last).



22. USING EXPRESS WITH SQL

22.1 Introduction to Sequelize ORM

- Sequelize is a tool that lets you talk to SQL databases (like MySQL/PostgreSQL) using JavaScript.

Install it:

```
bash
npm install sequelize mysql2
```

Create Sequelize instance:

```
js
const { Sequelize } = require('sequelize');
const sequelize = new Sequelize('db_name', 'user', 'password', {
  host: 'localhost',
  dialect: 'mysql' // or 'postgres'
});
```

22.2 Connecting to MySQL or PostgreSQL

```
js
sequelize.authenticate()
  .then(() => console.log('Connected to database'))
  .catch(err => console.error('Error:', err));
```

- This connects your Express app to SQL!

22.3 Defining Models

- Models are like tables in SQL.

```
js
const { DataTypes } = require('sequelize');

const User = sequelize.define('User', {
  name: DataTypes.STRING,
  email: DataTypes.STRING
});
```

- Sync model to create table:

```
js
sequelize.sync();
```



22. USING EXPRESS WITH SQL

22.4 Performing SQL CRUD Operations

- Create

```
js
User.create({ name: "John", email: "john@example.com" });
```

- Read

```
js
User.findAll().then(users => res.json(users));
```

- Update

```
js
User.update({ name: "Jane" }, { where: { id: 1 } });
```

- Delete

```
js
User.destroy({ where: { id: 1 } });
```



22. USING EXPRESS WITH SQL

22.4 Performing SQL CRUD Operations

- Create

```
js
User.create({ name: "John", email: "john@example.com" });
```

- Read

```
js
User.findAll().then(users => res.json(users));
```

- Update

```
js
User.update({ name: "Jane" }, { where: { id: 1 } });
```

- Delete

```
js
User.destroy({ where: { id: 1 } });
```




23. USING EXPRESS WITH TYPESCRIPT

23.1 Setting Up TypeScript in Express

- TypeScript helps us write safer JavaScript by adding types.

To start:

```
bash
npm install typescript ts-node-dev @types/node @types/express --save-dev
```

- Then create a tsconfig.json using:

```
bash
npx tsc --init
```

23.2 Typing req, res, and next

- In TypeScript, we write types like this:

```
js
import express, { Request, Response, NextFunction } from 'express';

const app = express();

app.get('/user/:id', (req: Request, res: Response, next: NextFunction) => {
  const userId: string = req.params.id;
  res.send(`User ID is ${userId}`);
});
```

- This helps catch mistakes before running the code.

23.3 TSConfig Best Practices

- Inside tsconfig.json, use:

```
json
{
  "compilerOptions": {
    "target": "es6",
    "module": "commonjs",
    "rootDir": "./src",
    "outDir": "./dist",
    "strict": true
  }
}
```

This will:

- Check for errors
- Keep code neat and safe
- Output JS files into dist/

23.4 Folder Structure in TS Projects

- Use a clear structure like:



23. USING EXPRESS WITH TYPESCRIPT

23.4 Folder Structure in TS Projects

- Use a clear structure like:

```
pgsql

project/
├── src/
│   ├── controllers/
│   ├── routes/
│   ├── models/
│   └── index.ts
├── dist/
├── tsconfig.json
└── package.json
```

- Use .ts files in src/, and compile them into JS in dist/.



24. ADVANCED API FEATURES

24.1 Pagination

- When you have lots of data, you show only a few items per page.

```
js
const page = parseInt(req.query.page) || 1;
const limit = parseInt(req.query.limit) || 10;
const start = (page - 1) * limit;

const users = await User.find().skip(start).limit(limit);
res.json(users);
```

24.2 Filtering and Searching

- Use query params to filter:

```
js
app.get('/products', async (req, res) => {
  const category = req.query.category;
  const products = await Product.find({ category });
  res.json(products);
});
```

24.3 Sorting with Query Parameters

- Sort by price, name, etc.:

```
js
const sortBy = req.query.sort || 'name';
const products = await Product.find().sort({ [sortBy]: 1 });
```

24.4 Handling Large Data Sets

Use:

- Pagination
- Indexes in the database
- Avoid sending unnecessary fields

You can also cache results to reduce database load.



25. ROLE-BASED ACCESS CONTROL (RBAC)

25.1 Admin vs User Role Checks

- You give users roles like user, admin, etc. in the database.

```
js
if (req.user.role === 'admin') {
  // allow access
}
```

25.2 Middleware for Role Verification

- Reusable middleware to check roles:

```
js
function checkRole(role) {
  return (req, res, next) => {
    if (req.user.role !== role) return res.status(403).send('Access Denied');
    next();
  };
}

// Usage
app.get('/admin', checkRole('admin'), (req, res) => {
  res.send("Welcome Admin");
});
```

25.3 Protecting Routes Based on Roles

You can protect:

- Admin-only routes
- Premium features
- Dashboard settings

Just check roles before allowing access.



26. TESTING EXPRESS APPLICATIONS

26.1 Unit Testing with Jest

Install:

```
bash
npm install jest ts-jest @types/jest --save-dev
```

In a test file:

```
ts
test('2 + 2 = 4', () => {
  expect(2 + 2).toBe(4);
});
```

Run:

```
bash
npx jest
```

26.2 Integration Testing with Supertest

Install:

```
bash
npm install supertest @types/supertest --save-dev
```

Example:

```
ts
import request from 'supertest';
import app from './app';

test('GET /', async () => {
  const res = await request(app).get('/');
  expect(res.statusCode).toBe(200);
});
```

26.3 Testing Routes and Controllers

Write tests to check:

- GET returns data
- POST creates item
- PUT updates
- DELETE removes item

26.4 Mocking Database and JWT

Instead of using a real DB:

- Use `jest.mock()` to fake database functions

For example:



26. TESTING EXPRESS APPLICATIONS

26.4 Mocking Database and JWT

For example:

```
jest.mock('../models/User', () => ({
  find: jest.fn().mockResolvedValue([{ name: 'Test' }])
}));
```

Same with JWT:

```
jest.mock('jsonwebtoken', () => ({
  verify: () => ({ id: '123', role: 'admin' })
}));
```



27. WEBSOCKETS WITH EXPRESS

27.1 What are WebSockets?

- WebSockets allow two-way communication between the server and client in real-time.
- Normal websites: client asks, server responds.
- With WebSockets: they can talk anytime, without reloading.

Useful for:

- Chat apps
- Live updates
- Multiplayer games

27.2 Using socket.io with Express

Install socket.io:

```
bash
npm install socket.io
```

Set up in your server:

```
js
const express = require('express');
const http = require('http');
const socketIO = require('socket.io');

const app = express();
const server = http.createServer(app);
const io = socketIO(server);

io.on('connection', (socket) => {
  console.log('User connected');
});

server.listen(3000);
```

27.3 Real-time Messaging Example

```
js
io.on('connection', (socket) => {
  socket.on('message', (msg) => {
    console.log('Received:', msg);
    io.emit('message', msg); // send to all users
  });
});
```

Client side:

```
html
<script src="/socket.io/socket.io.js"></script>
<script>
  const socket = io();
  socket.emit('message', 'Hello!');
  socket.on('message', (msg) => alert(msg));
</script>
```



27. WEBSOCKETS WITH EXPRESS

27.4 Socket Events and Rooms

- You can send messages to specific rooms:

```
socket.join('room1');  
socket.to('room1').emit('message', 'Hi Room 1');
```

- Useful for private chats or group chats.



28. CODE STRUCTURE AND BEST PRACTICES

28.1 Folder Structure for Large Projects

- A clean structure makes it easy to manage code:

```
pgsql

project/
├── controllers/
├── routes/
├── models/
├── services/
├── middlewares/
├── utils/
├── app.js
└── server.js
```

28.2 Service vs Controller Layer

- Controllers: Handle HTTP request/response
- Services: Handle logic and database

Example:

```
js

// controller
const userService = require('../services/userService');
exports.getUser = async (req, res) => {
  const user = await userService.findUser(req.params.id);
  res.json(user);
};

// service
exports.findUser = (id) => User.findById(id);
```

28.3 Using Async/Await Everywhere

- Always use async/await to avoid .then() and .catch() chains:

```
js

app.get('/', async (req, res) => {
  try {
    const users = await getUsers();
    res.json(users);
  } catch (err) {
    res.status(500).send('Error');
  }
});
```

28.4 Avoiding Callback Hell

- Instead of:

```
js

fs.readFile('file', (err, data) => {
  db.save(data, (err, result) => {
    sendEmail(result, () => {
      res.send('Done');
    });
  });
});
```



28. CODE STRUCTURE AND BEST PRACTICES

28.4 Avoiding Callback Hell

- Use:

```
js

const data = await fs.promises.readFile('file');
const result = await db.save(data);
await sendEmail(result);
res.send('Done');
```



29. POPULAR EXPRESS PACKAGES

Here are must-know packages:

Basic Middleware

- cors – Allows cross-origin access
- helmet – Adds security headers
- morgan – Logs all requests
- dotenv – Loads environment variables

```
bash  
npm install cors helmet morgan dotenv
```

API & Validation

- express-validator – Validate form inputs
- jsonwebtoken – Create & verify tokens
- multer – Upload files

Utilities

- compression – Makes responses smaller
- cookie-parser – Parses cookies
- express-rate-limit – Limits request rate



30. COMMON MISTAKES BEGINNERS MAKE IN EXPRESS.JS

1. Forgetting to Start the Server

- **Mistake:** You write the whole app but forget `app.listen()`.

```
// Missing this:
app.listen(3000, () => {
  console.log("Server started");
});
```

2. Not Using Middleware Correctly

- **Mistake:** Not calling `next()` in custom middleware.

```
function myMiddleware(req, res, next) {
  console.log("Check");
  // Forgot next() - request will hang
}
```

3. Not Handling Errors Properly

- **Mistake:** Ignoring errors or not sending a response back.

```
app.get('/', (req, res) => {
  throw new Error("Oops!");
  // App crashes
});
```

- You should use a central error handler.

4. Mixing Logic Inside Routes

- **Mistake:** Writing database code and response in one file.
- **Better:** Move DB logic to a service file.

5. Hardcoding Secrets

Mistake:

```
const jwtSecret = "my-secret-key"; // BAD
```

- Store it in a `.env` file.

6. Not Using `.env` Files

- **Mistake:** Putting passwords or MongoDB URLs directly in the code.
- **Use:**

```
DB_URL=mongodb+srv://user:pass@cluster
```



30. COMMON MISTAKES BEGINNERS MAKE IN EXPRESS.JS

6. Not Using .env Files

- Then in code:

```
require('dotenv').config();
const db = process.env.DB_URL;
```

7. Forgetting to Parse Request Body

- **Mistake:** Not using `express.json()` or `express.urlencoded()`.

```
app.use(express.json()); // Required to read req.body
```

8. Using Wrong HTTP Methods

- **Mistake:** Using GET when you want to update data.

Use:

- GET → Read
- POST → Create
- PUT → Update
- DELETE → Delete

9. Ignoring Async/Await or Promises

- **Mistake:** Not handling async properly.

```
app.get('/', (req, res) => {
  const users = User.find(); // Missing await
});
```

- Use:

```
app.get('/', async (req, res) => {
  const users = await User.find();
  res.json(users);
});
```

10. Not Using Route Files and Routers

- **Mistake:** All routes in `app.js` file.
- Create separate files like:

```
routes/userRoutes.js
routes/blogRoutes.js
```

- Use `express.Router()`.



30. COMMON MISTAKES BEGINNERS MAKE IN EXPRESS.JS

11. Missing return in Middleware

- **Mistake:**

```
js
if (!user) {
  res.status(401).send("Unauthorized");
  // No return - rest of code still runs
}
```

- Add return after response.

12. Not Validating User Input

- **Mistake:** Allowing any data into the database.
- Use express-validator to check data before saving.

13. Not Handling 404 Errors

- **Mistake:** If no route matches, nothing happens.
- Add this at the bottom:

```
js
app.use((req, res) => {
  res.status(404).send("Page not found");
});
```

14. Not Securing API

- **Mistake:** Not using security packages like:

```
js
const helmet = require("helmet");
const cors = require("cors");

app.use(helmet());
app.use(cors());
```

15. Not Using Proper Status Codes

- **Mistake:** Sending 200 (OK) even for errors.

Use:

- 200 → Success
- 400 → Bad request
- 401 → Unauthorized
- 404 → Not found
- 500 → Server error

16. Not Closing Database Connections

- For long-running apps, make sure DB connections are properly handled and closed if needed.



30. COMMON MISTAKES BEGINNERS MAKE IN EXPRESS.JS

17. Forgetting next() in Middleware

- Without next(), request doesn't move forward and hangs forever.

18. Overusing Global Middleware

- Using app.use() for all routes even when it's not needed.
- Use middleware only where required.

19. Not Handling File Uploads Carefully

- Uploading big files without limits can crash the server.
- Use multer and set size limits.

20. Not Using try/catch in Async Functions

- If you don't use try/catch, errors crash your app.
- Always wrap async logic:

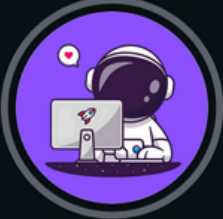
```
js
app.get('/', async (req, res) => {
  try {
    const data = await getData();
    res.json(data);
  } catch (err) {
    res.status(500).send("Error occurred");
  }
});
```



Was this post helpful ?

 **coders_section** 



Coding | Programming | HTML | CSS

130
posts

17K
followers

92
following



Follow Our 2nd Account

 **coders.100**  



Coding • HTML • CSS • JAVASCRIPT

5
posts

31
followers

4
following



Follow For More



[Tap Here](#)